.

Lesson 08 Demo 02

Testing the React Component Using Jest

Objective: To set up a React project, create a simple component, and test its behavior using Jest and the React Testing Library to validate functionality and interactions

Tools required: Visual Studio Code, Node.js, and npm

Prerequisites: A good understanding of JavaScript, HTML, React, and DOM (document object model), as well as some knowledge of Jest

Steps to be followed:

1. Set up a new React project and install Jest
2. Create a simple React component
3. Write a Jest test for the Counter component
4. Configure the Jest
5. Run the Jest test

Step 1: Set up a new React project and install Jest

1.1 Use the following command to open the terminal and create a React project:
npx create-react-app react-jests-example

```
labuser@ip-172-31-31-96:~$ npx create-react-app react-jests-example
Need to install the following packages:
create-react-app@5.1.0
Ok to proceed? (y) y

npm warn deprecated uid-number@0.0.6: This package is no longer supported.
npm warn deprecated inflight@1.0.6: This module is not supported, and leaks memory. Do not use it.
ce async requests by a key value, which is much more comprehensive and powerful.
npm warn deprecated rimraf@2.7.1: Rimraf versions prior to v4 are no longer supported
npm warn deprecated fstream-ignore@1.0.5: This package is no longer supported.
npm warn deprecated glob@7.2.3: Glob versions prior to v9 are no longer supported
npm warn deprecated fstream@1.0.12: This package is no longer supported.
npm warn deprecated tar@2.2.2: This version of tar is no longer supported, and will not receive se
create-react-app is deprecated.
```

.

```
  npm start
    Starts the development server.

  npm run build
    Bundles the app into static files for production.

  npm test
    Starts the test runner.

  npm run eject
    Removes this tool and copies build dependencies, configuration files
    and scripts into the app directory. If you do this, you can't go back!

We suggest that you begin by typing:

  cd react-jests-example
  npm start

Happy hacking!
```

1.2 Using the following commands, navigate to the **react-jests-example** folder and then install Jest:

cd react-jests-example

npm install --save-dev jest

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    COMMENTS

labuser@ip-172-31-31-96:~$ cd react-jests-example/
labuser@ip-172-31-31-96:~/react-jests-example$
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    COMMENTS

labuser@ip-172-31-31-96:~/react-jests-example$ npm install --save-dev jest

up to date, audited 1343 packages in 18s
```

.

Step 2: Create a simple React component

2.1 Use the following command to navigate to the **src** folder:

cd src

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    COMMENTS
labuser@ip-172-31-31-96:~/react-jests-example$ pwd
/home/labuser/react-jests-example
labuser@ip-172-31-31-96:~/react-jests-example$ ls
README.md  node_modules  package-lock.json  package.json  public  src
labuser@ip-172-31-31-96:~/react-jests-example$ cd src
labuser@ip-172-31-31-96:~/react-jests-example/src$
```

2.2 Use the following command to create a new component file named **Counter.js** in the src folder:

nano Counter.js

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    COMMENTS
labuser@ip-172-31-31-96:~/react-jests-example/src$ nano Counter.js
```

2.3 Add the following code to the Counter.js file, press Ctrl+S to save, and then press Ctrl+X to exit from the nano editor:

```
import React, { useState } from 'react';
function Counter() {
  const [count, setCount] = useState(0);

  const increment = () => {
    setCount(count + 1);
  };

  const decrement = () => {
    setCount(count - 1);
  };

  return (
    <div>
      <h1>Counter</h1>
```

```
    <p>Count: {count}</p>
    <button onClick={increment}>Increment</button>
    <button onClick={decrement}>Decrement</button>
   </div>
  );
}
export default Counter;
```

```
  GNU nano 6.2                                                    Counter.js *
import React, { useState } from 'react';
function Counter() {
  const [count, setCount] = useState(0);

  const increment = () => {
    setCount(count + 1);
  };

  const decrement = () => {
    setCount(count - 1);
  };

  return (
    <div>
      <h1>Counter</h1>
      <p>Count: {count}</p>
      <button onClick={increment}>Increment</button>
      <button onClick={decrement}>Decrement</button>
```

Step 3: Write a Jest test for the Counter component

3.1 Use the following command to create a test file named **Counter.test.js** in the src folder:

nano Counter.test.js

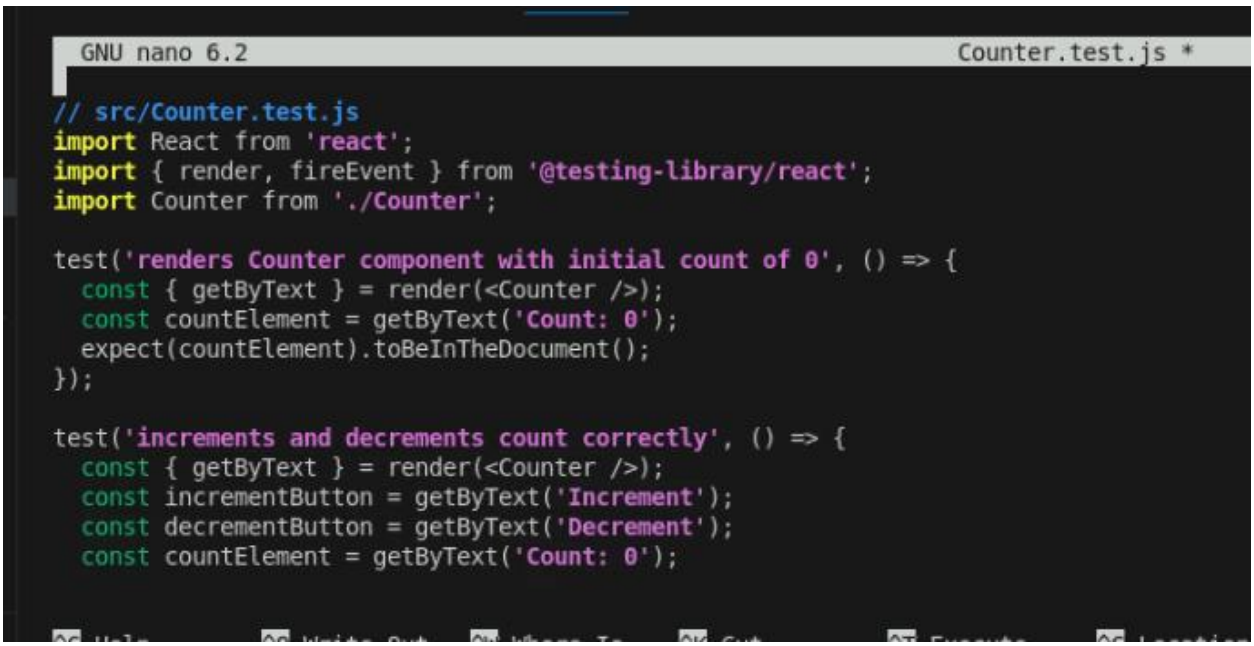
```
labuser@ip-172-31-31-96:~/react-jests-example/src$ nano Counter.test.js
labuser@ip-172-31-31-96:~/react-jests-example/src$
```

3.2 Add the following code in the Counter.test.js file, press Ctrl+S to save, and then press Ctrl+X to exit from the nano editor:

```
// src/Counter.test.js
import React from 'react';
import { render, fireEvent } from '@testing-library/react';
import Counter from './Counter';

test('renders Counter component with initial count of 0', () => {
  const { getByText } = render(<Counter />);
  const countElement = getByText('Count: 0');
  expect(countElement).toBeInTheDocument();
});
```

.

```
test('increments and decrements count correctly', () => {
  const { getByText } = render(<Counter />);
  const incrementButton = getByText('Increment');
  const decrementButton = getByText('Decrement');
  const countElement = getByText('Count: 0');

  fireEvent.click(incrementButton);
  expect(countElement).toHaveTextContent('Count: 1');

  fireEvent.click(decrementButton);
  expect(countElement).toHaveTextContent('Count: 0');
  });
```



.

Step 4: Configure the Jest

4.1 Create a file named **jest.config.js** in the react-jest-example directory using the following command:
nano jest.config.js

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    COMMENTS

labuser@ip-172-31-31-96:~/react-jests-example$ nano jest.config.js
```

4.2 Copy the code given below and paste it into the file **jest.config.js** to specify the test environment as jsdom:

```
// jest.config.js
module.exports = {
  testEnvironment: 'jsdom',
};
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    COMMEN

  GNU nano 6.2

// jest.config.js
module.exports = {
  testEnvironment: 'jsdom',
};
```

Step 5: Run the Jest test

5.1 Run the test using the following command:
npm test

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    GITLENS    SQL CONSOLE

labuser@ubuntu2204:~/react-jests-example$ npm test
```

.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    COMMENTS

 PASS  src/Counter.test.js
 PASS  src/App.test.js

Test Suites: 2 passed, 2 total
Tests:       3 passed, 3 total
Snapshots:   0 total
Time:        2.44 s
Ran all test suites.

Watch Usage
 › Press f to run only failed tests.
 › Press o to only run tests related to changed files.
 › Press q to quit watch mode.
 › Press p to filter by a filename regex pattern.
 › Press t to filter by a test name regex pattern.
 › Press Enter to trigger a test run.
```

Note: Jest will discover and execute your tests. If everything is set up correctly, you will see an output indicating that the test passed.

By following these steps, you have successfully set up a React project, created a simple component, and tested its behavior using Jest and the React Testing Library. This demonstration ensures that your React components function correctly and handle interactions as expected, improving the reliability and maintainability of your application.